EXECUTIVE SUMMARY

Target: https://vuln-example.com/api/v1

Scan ID: PRO-VERIFICATION-777

Scan Date: 2026-03-31 19:43:18

Scan Duration: N/A

Total Requests Sent: 1500

Avg Latency: 120 ms

Peak Concurrency: 10

Findings: 2 vulnerabilities detected

Severity Breakdown: Critical: 1 | High: 0 | Medium: 1 | Low: 0

AI Inference Calls: 2

LLM Avg Latency: N/A

Circuit Breaker Activations: 0

- Overall Risk**: High due to potential security vulnerabilities in the CortexEngine API.
- Most Critical Category**: Critical as it involves unauthorized access and data manipulation.
- Immediate Actions**: Implementing robust authentication mechanisms, enhancing API rate limits, and regularly updating software to patch known vulnerabilities.
- Long-Term Recommendations**: Encouraging regular updates of the CortexEngine API to mitigate future threats.

EXECUTIVE STRATEGIC IMPACT

CortexEngine is vulnerable to SQL Injection and Cross-Site Scripting, which can lead to unauthorized access to sensitive data or execute arbitrary code on the target system. To achieve a high-impact objective, an attacker might chain these vulnerabilities together by leveraging multiple injection points or exploiting specific vulnerabilities in the application's database schema.

DETAILED FINDINGS

FILTER: INJECTION & FUZZING

Finding #1: SQL Injection

CRITICAL

CWE: CWE-89

CVSS Score: 9.8 (CRITICAL)

THREAT SCORE: 98/100

Description:

- The endpoint accepts user input without proper sanitization or parameterization, allowing raw SQL fragments to be executed.
- When the payload ' OR '1'='1 is submitted in a user field, the backend SQL query evaluates to a tautology, returning all user records instead of filtering by the intended criteria.
- The vulnerability exists due to dynamic SQL construction using string concatenation and absence of prepared statements or input validation.

Impact:

- Strategic Impact: Compromise of user authentication systems undermines the entire platform's trust model and exposes the organization to reputational damage.
- Financial Impact: Potential fines under data protection regulations, cost of breach response, and loss of business due to service downtime or customer attrition.
- Technical Impact: Full database read/write access possible, enabling privilege escalation, data exfiltration, and potential lateral movement within the network.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'SQL_INJECTION' was intercepted to protect system integrity.

FORENSIC ANALYSIS

```
Method: GET | Param: N/A
```

URL: https://vuln-example.com/api/v1/user
Analysis: The application directly concatenates user input into SQL queries without sanitization or parameterization.

Table 1: Payload Decomposition

Component	Value	Technical Function
Target ID	' OR '1'='1	The application directly concatenates us
Evidence	DB Error: Syntax error ne	The server's DB Error response explicitl
Result	200	An attacker can manipulate database quer

Payload Specifications:

```
Vector Category: SQL Injection
Raw Payload:     ' OR '1'='1
Encoded:         27204f52202731273d2731
Encoding Type:   None
```

Reproduction Command:

```
curl -X GET 'https://vuln-example.com/api/v1/user'
```

HTTP Traffic Snapshot:

Request:

```
GET https://vuln-example.com/api/v1/user HTTP/1.1
Host: vuln-example.com
{}

' OR '1'='1
```

Response:

```
HTTP/1.1 200
Content-Type: application/json

DB Error: Syntax error near '' OR '1'='1'
```

Remediation:

- Use parameterized queries or prepared statements with bound variables for all database interactions.
- Implement a Web Application Firewall (WAF) with SQLi signature detection and enforce input validation via allowlists at the API gateway.
- Deploy real-time logging and alerting for SQL error patterns, unusual query volumes, or repeated failed authentication attempts from single IPs.

Recommended Code Fix:

```
def secure_function(username):
  cursor.execute("SELECT * FROM users WHERE username = %s", (username,))
```

FILTER: INFECTION & FUZZING

FILTER: INFECTION & FUZZING

Finding #2: Cross-Site Scripting (XSS)

MEDIUM

CWE: CWE-79

CVSS Score: 5.1 (MEDIUM)

THREAT SCORE: 51/100

Description:

- A reflected XSS vulnerability was observed when the <script>alert(1)</script> payload was submitted via the q parameter in the search endpoint.
- The endpoint reflected the unescaped payload directly in the HTTP response, causing the browser to execute the script in the context of the victim's session.
- The vulnerability exists due to lack of input sanitization and output encoding on the server-side response generation for search queries.

Impact:

- Strategic Impact: Compromised user trust and potential brand erosion due to unauthorized client-side code execution.
- Financial Impact: Risk of regulatory penalties (e.g., GDPR, CCPA) and costs associated with incident response and customer notification.
- Technical Impact: Session hijacking, data exfiltration, and potential lateral movement within the application if combined with other flaws.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'CROSS_SITE_SCRIPTING' was intercepted to protect system integrity.

FORENSIC ANALYSIS

```
Method: GET | Param: N/A
URL: https://vuln-example.com/api/v1/search?q=<script>alert(1)</script>
Analysis: The server improperly reflects user-supplied input in the response without proper encoding or sanitization.
```

Table 1: Payload Decomposition

Component	Value	Technical Function
Target ID	<script>alert(1)</script>	The server improperly reflects user-supp
Evidence	Script reflected in respo	The server returned the exact payload <s
Result	200	An attacker can execute arbitrary JavaSc

Payload Specifications:

```
Vector Category: Cross-Site Scripting (XSS)
Raw Payload:     <script>alert(1)</script>
Encoded:         3c7363726970743e616c6572742831293c2f7363
Encoding Type:   None
```

Reproduction Command:

```
curl -X GET 'https://vuln-example.com/api/v1/search?q=<script>alert(1)</scr
```

HTTP Traffic Snapshot:

Request:

```
GET https://vuln-example.com/api/v1/search?q=<script>alert(1)</script> HTTP/1.1
Host: vuln-example.com
{}

<script>alert(1)</script>
```

Response:

```
HTTP/1.1 200
Content-Type: application/json

Script reflected in response body
```

Remediation:

- Implement strict output encoding (e.g., HTML entity encoding) for all user-supplied data reflected in HTTP responses.
- Deploy a Content Security Policy (CSP) with script-src 'self' to mitigate impact of any residual XSS flaws.
- Monitor for anomalous search queries containing script tags or encoded JavaScript using WAF rules and SIEM correlation.

Recommended Code Fix:

```
def secure_function():
  query = request.args.get('q', '')
  return escape(query)  # Use context-aware HTML escaping library
```

SCAN TIMELINE

```
[Orchestrator] VULN_CONFIRMED -> https://vuln-example.com/api/v1/user - 2026-03-31 19:43:18
[Orchestrator] VULN_CONFIRMED -> https://vuln-example.com/api/v1/user - 2026-03-31 19:43:18
[Orchestrator] VULN_CONFIRMED -> https://vuln-example.com/api/v1/search?q - 2026-03-31 19:43:18
```